Звіт до лабораторної роботи. Варіант 8 — replace_if

Виконала Докова Вікторія, К-28

Мета роботи

Дослідити швидкодію алгоритму std::replace_if для масивів різного розміру. Порівняти звичайний алгоритм, різні політики виконання та власний паралельний алгоритм. Визначити найкраще значення K, дослідити залежність часу від кількості потоків і порівняти результати з оптимізацією /O2 та без оптимізації /Od.

Як працює програма

Програма написана на C++20 і зібрана у Visual Studio 2022 (MSVC, x64, Release). Для дослідження використовуються масиви розміром 10 000, 100 000, 1 000 000 і 10 000 000 елементів. Числа генеруються за допомогою std::mt19937 і uniform_int_distribution у діапазоні від 1 до 1 000 000 та записуються у файли txt. Якщо файли вже існують, вони не перезаписуються, тому для обох рівнів оптимізації використовуються однакові дані.

Використовуються два предикати: перший перевіряє, чи менше число за 500 000, другий — чи є число простим. Алгоритм замінює на 0 усі елементи, для яких предикат повертає true.

Кожен вимір виконується п'ять разів, береться медіана. Перед кожним запуском створюється нова копія масиву, бо replace_if його змінює; час копіювання не враховується. Після кожного запуску результат перевіряється на правильність.

Власний алгоритм ділить масив на K приблизно однакових частин, кожна обробляється окремим потоком за допомогою std::replace_if. Після завершення всіх потоків результат уже готовий, бо частини не перетинаються і об'єднання не потрібне.

Результати

Процесор: Intel Core Ultra 7 155H, 22 апаратні потоки. Компілятор: MSVC 19.44, Visual Studio 2022.

Масив з 10 000 000 елементів, /O2, дешевий предикат: без політики — 65,3 мс, seq — 66,5 мс, unseq — 67,4 мс, par — 6,8 мс, par_unseq — 6,5 мс. Власний алгоритм: K = 1 — 66,1 мс, найкраще K = 20 — 8,5 мс, K = 704 — 56,2 мс.

Дорогий предикат: без політики — 993 мс, seq — 941 мс, unseq — 931 мс, par — 94,3 мс, par_unseq — 86,8 мс. Власний алгоритм: K = 1 — 954 мс, найкраще K = 40 — 101,8 мс.

Масив з 10 000 елементів, дешевий предикат: без політики — 0,051 мс, par — 0,099 мс, власний алгоритм при K = 1 — 0,263 мс, при K = 704 — 41,3 мс.

Без оптимізації (/Od) для масиву з 10 000 000 елементів без політики: дешевий предикат — 93,3 мс, дорогий — 1045 мс.

Висновки

1. Найкраще K близьке до кількості апаратних потоків: K = 20 для дешевого і K = 40 для дорогого предиката при 22 потоках (K / P = 0,91 і 1,82). Коли K стає більшим, виграшу вже немає: потоків більше, ніж процесор може виконувати одночасно.
2. Коли K більше за кількість потоків, час зростає лінійно: кожен новий потік треба створити і дочекатися його завершення, це приблизно 58 мкс на потік. При подвоєнні K з 352 до 704 час також зростає майже вдвічі.
3. Політики par і par_unseq прискорили обробку великих масивів приблизно в 10 разів. Політики seq і unseq майже не дали прискорення.
4. На малих масивах паралельна обробка невигідна: створення потоків займає більше часу, ніж сама робота, тому par і власний алгоритм там повільніші за звичайний виклик.
5. Оптимізація /O2 прискорює роботу. Для дешевого предиката різниця помітна, а для дорогого майже відсутня, бо там більшість часу займає ділення, яке оптимізація не прискорює.